

CROSSY ROAD

An OOP-Based Infinite Runner Game

CT-260 Object Oriented Programming · NED University of Engineering & Technology
Department of Computer Science & IT · Spring 2026 · Submitted May 2026
Instructor: [Dr Murk Maurvi]

TEAM : B Y T E M E

CT-25055 **Zuha Azhar** — Vehicles, World Gen & Game State
CT-25070 **Areeba** — Compiler, Logs, Sprites & Collision
CT-25087 **Muhammad Ashar Hussain** — Game Loop, UI & Audio
CT-25098 **Ali Hasan Farooqui** — Static Obstacles, UI & UML

ABSTRACT

This report documents the design, architecture, and implementation of Crossy Road — an infinite tile-based runner game developed in C++ using the SFML graphics library, submitted as a Complex Computing Problem (CCP) for CT-260 Object Oriented Programming at NED University of Engineering & Technology.

The game recreates core mechanics of the original Crossy Road mobile game: a player navigates a procedurally generated, infinitely scrolling world of safe grass zones, vehicle-filled roads, and log-laden rivers. The primary focus is rigorous application of object-oriented software design principles to produce a maintainable, extensible, well-structured codebase. The Prototype and Game State design patterns are applied and analyzed in detail.

6	4	3	2	4
ENTITY TYPES	COLLISION TYPES	GAME STATES	DESIGN PATTERNS	TEAM MEMBERS

1. PROBLEM STATEMENT

Crossy Road is a tile-based, infinite-runner game inspired by the globally popular mobile game of the same name. The player controls a character navigating an endlessly scrolling world of procedurally generated safe zones, traffic roads, and rivers — avoiding vehicles, drowning, and riding logs to cross water. The game presents a non-trivial computing problem requiring the modelling of a dynamic, continuously evolving world populated by multiple interacting entity types, rendered in real time via a graphical window.

CORE COMPUTING SUB-PROBLEMS

- ▶ Infinite World Management — content procedurally spawned ahead of the camera and despawned behind it, every frame, without memory leaks or performance degradation.
- ▶ Heterogeneous Entity Modeling — six distinct entity types (Bike, Car, Dumper, Log, Tree, Rock) sharing common properties but exhibiting entirely different behaviors.
- ▶ Multiple Collision Semantics — four structurally different collision types: physical blocking, lethal contact, platform riding, and environmental death (drowning without a log).
- ▶ State Machine Management — clean transitions between MENU, PLAYING, and DEAD states, each requiring different input handling, update logic, and rendering.
- ▶ Persistent Data — high score surviving across game sessions via file I/O with graceful handling of missing files on first run.

This problem maps naturally and compellingly to OOP because the game world directly models real-world entities with well-defined identities, attributes, and behaviors. Every OOP concept covered in CT-260 finds a direct, non-artificial application in this project.

2. OOP PRINCIPLES IDENTIFICATION

All major OOP principles from the CT-260 syllabus are demonstrated directly in the Crossy Road codebase. The grid below maps each principle to its concrete implementation:

ENCAPSULATION Access-controlled members protect object state across all classes	INHERITANCE Obstacle → Vehicle, Log, Tree, Rock hierarchy shares common interface	POLYMORPHISM Virtual dispatch drives update() / draw() / clone() for all entity types
ABSTRACTION Public APIs hide implementation — WorldGenerator behind one method call	COMPOSITION WorldGenerator owns prototype registry; Obstacle owns Sprite & Texture	DYNAMIC BINDING vtable routes obs->update(dt) to correct subclass override at runtime

ENCAPSULATION

§ 2.1

INHERITANCE

§ 2.2

2.1 ENCAPSULATION

In the Obstacle class, core sprite pointer, texture, speed, and direction fields are declared protected — accessible to subclasses but invisible to the game loop or collision system. External code interacts exclusively through the public interface: getBounds(), getSpeedX(), setPosition(), setSpeed(), and the virtual update()/draw() methods.

The WorldGenerator class presents the most powerful application: its entire internal state — prototype registry, RNG engine, highestLaneY cursor, and lanesSinceLastSafe counter — is private. The entire subsystem is hidden behind one public method: update(cameraY, lanes, obstacles).

2.2 INHERITANCE

The abstract Obstacle base class captures all state and behavior common to every game entity. Declaring `update()` and `draw()` as pure virtual makes the class abstract — it cannot be instantiated directly, enforcing the rule that only concrete, well-defined entities exist in the game world.

SUBCLASS	CATEGORY	KEY SPECIALIZATION
Vehicle	Moving	Horizontal movement, screen wrapping, three variants via <code>VehicleType</code> enum
Log	Moving (Platform)	Horizontal movement, wrapping, provides velocity to player when ridden
Tree	Static	No movement; blocks player via static collision check in <code>Collision</code> class
Rock	Static	No movement; blocks player via static collision check in <code>Collision</code> class

POLYMORPHISM

§ 2.3

ABSTRACTION

§ 2.4

COMPOSITION

§ 2.5

DYNAMIC BINDING

§ 2.6

2.3 POLYMORPHISM

The obstacles vector stores `Obstacle*` pointers. The frame update loop calls `obs->update(dt)` and `obs->draw(window)` — the vtable ensures the correct `Vehicle`, `Log`, `Tree`, or `Rock` implementation is dispatched at runtime. Without this, every entity operation would require a switch-case block that breaks with each new type added.

The `Collision` class also uses `dynamic_cast<Vehicle*>` and `dynamic_cast<Log*>` for targeted collision semantics while still receiving all obstacles through the uniform `Obstacle*` interface. The virtual `clone()` method additionally enables the Prototype pattern.

2.4 ABSTRACTION

The `Obstacle` abstract class defines the what — `update`, `draw`, `clone` — without specifying the how. `WorldGenerator` hides an entire procedural generation subsystem behind a single method call. The `Collision` class hides four completely different detection algorithms behind four named static methods. `AudioManager` hides SFML's buffer system behind `playSound(name)`.

2.5 COMPOSITION

`WorldGenerator` composes a prototype registry (`std::map<string, Obstacle*>`) — it owns these objects entirely, creating them in `registerPrototypes()` and deleting them in its destructor. `Obstacle` composes an SFML `Sprite*` and `Texture`, managing the sprite's full lifetime across construction, destruction, and deep-copy.

2.6 DYNAMIC BINDING

Every Obstacle subclass overrides the pure virtual update(), draw(), and clone(). The C++ runtime maintains a vtable for each class. When obs->update(dt) is called, the vtable lookup dispatches to the correct override — without explicit branching. Dynamic binding also makes the Prototype pattern work: clone() on an Obstacle* correctly returns a Vehicle*, Log*, Tree*, or Rock* — impossible with static binding.

3. SYSTEM DESIGN & ARCHITECTURE

The system is organized into four well-separated architectural layers, each with a clearly defined responsibility. Higher layers depend on lower layers but not the reverse, minimizing coupling.

LAYER	COMPONENTS	RESPONSIBILITY
Entity Layer	Obstacle, Vehicle, Log, Tree, Rock, Lane	All game-world objects. Manages own state, textures, and rendering via a uniform Obstacle interface.
World Layer	WorldGenerator	Procedural generation of an infinite scrolling world. Owns the prototype registry, manages spawn/despawn, enforces lane type rules.
Logic Layer	Collision	All collision detection and response. Stateless static utility — takes game world snapshots and returns results.
Application Layer	main.cpp	Window management, event loop, camera, player movement, game state machine, HUD rendering, score, audio triggering.

3.1 CLASS RESPONSIBILITIES

ENTITY LAYER	Obstacle (Abstract Base) — Declares the contract for all game entities: pure virtual update(float dt), draw(RenderWindow&), and clone().
WORLD LAYER	Provides concrete implementations of getBounds(), getSpeedX(), setPosition(), setSpeed(), applyFlip(). The copy constructor performs a deep copy of the Sprite — essential for the Prototype pattern.
LOGIC LAYER	Vehicle — Three variants (BIKE, CAR, DUMPER) via VehicleType enum. Static textureFor() and sizeFor() methods eliminate duplicate constructor arguments. update() applies horizontal movement scaled by delta time and wraps around screen edges. Log — Structurally similar to Vehicle but semantically a platform. The game loop reads log->getSpeedX() and moves the player by that velocity each frame when riding. Tree / Rock — Static subclasses with empty update(). Exist to provide correct runtime types for dynamic_cast in Collision, and to block player movement via checkStaticBlocking().

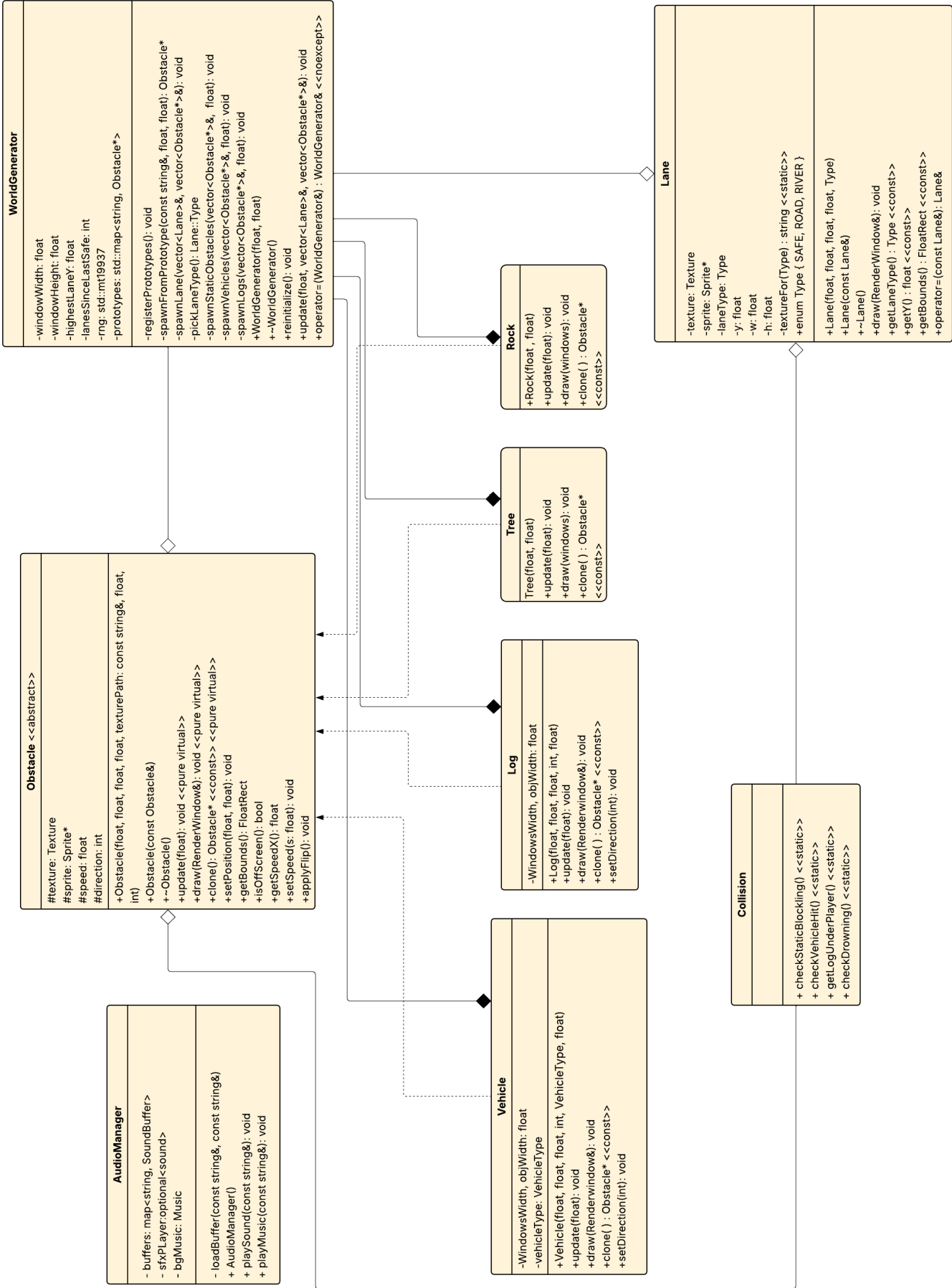
Lane — A single horizontal strip with type SAFE (green), ROAD (grey), or RIVER (blue). WorldGenerator reads getLaneType() to determine what obstacles to spawn per row.

WorldGenerator — Maintains highestLaneY cursor. Each frame compares against a spawn threshold and calls spawnLane(). The lanesSinceLastSafe counter enforces a maximum of 4 consecutive non-safe lanes, preventing unplayable stretches.

Collision (Static Utility) — Four static methods: checkStaticBlocking (simulates future position before commit), checkVehicleHit (AABB per frame), getLogUnderPlayer (platform riding), checkDrowning (river + no log check). Each handles a semantically distinct collision algorithm.

3.2 UML CLASS RELATIONSHIPS

CLASS	RELATIONSHIP	TARGET	MEANING
Vehicle, Log, Tree, Rock	Inheritance (is-a)	Obstacle	Share base interface; override update/draw/clone
WorldGenerator	Composition (owns)	map<string, Obstacle*>	Owns and manages all prototype instances
WorldGenerator	Association (uses)	Lane, Obstacle*	Creates and modifies lanes/obstacles each frame
Collision	Dependency (uses)	Obstacle*, Lane, Log*	Receives references; does not own any objects
main.cpp	Aggregation (holds)	vector<Lane>, vector<Obstacle*>	Manages world object lifetime in game loop
Obstacle	Composition (owns)	Sprite*, Texture	Manages SFML rendering objects internally



4. SOLUTION DEFINITION

4.1 GAME LOOP ARCHITECTURE

The main game loop follows the industry-standard Event → Update → Draw cycle, executed every frame while the SFML window is open. All operations are gated behind the current GameState. The use of `clock.restart().asSeconds()` as a delta-time multiplier ensures frame-rate independence.

GAME LOOP PHASES

- ▶ Event Phase — polls all window events; arrow keys trigger player movement with pre-checked collision; Enter restarts from MENU or DEAD.
- ▶ Update Phase — advances cameraY, calls `worldGen.update()`, calls `obs->update(dt)` for all obstacles, processes log riding, scores new rows crossed, checks all death conditions.
- ▶ Draw Phase — sets camera view for world rendering (lanes, obstacles, player), then switches back to default view for fixed HUD (score, high score text).

4.2

World Gen

4.2 INFINITE WORLD GENERATION

WorldGenerator maintains a floating-point cursor, `highestLaneY`, initialized to the bottom of the screen. Each frame, a spawn threshold is computed; while `highestLaneY` exceeds it, `spawnLane()` generates a new row. Lane type selection enforces a safety guarantee: after 4 consecutive non-safe lanes, the next is always SAFE. Despawning removes content below the camera's bottom edge, deleting obstacle pointers to prevent leaks.

4.3

Collision

4.3 COLLISION SYSTEM

Sprites & Audio

4.5
Score

4.6
Camera

TYPE	METHOD	RESULT
Static Block	Simulate future FloatRect before move commits	Cancel move — player stays put
Vehicle Hit	Current FloatRect vs. all Vehicle instances	GameState → DEAD, death sound
Log Riding	Current FloatRect vs. Log instances	<code>player.move(log.getSpeedX() * dt)</code>
Drowning	RIVER lane + no Log intersection	GameState → DEAD, death sound

4.4 SPRITE & AUDIO SYSTEM

Each entity type loads its texture in `assets/images/` during construction, with graceful fallback to a 1×1 white image if missing. Direction-aware sprite flipping in `Obstacle::applyFlip()` negates X scale and shifts origin — producing a mirrored image without a second texture asset.

AudioManager uses `std::optional<Sound>` to avoid SFML 3's requirement that Sound requires an associated buffer at construction. Background music loops at reduced volume; hop/death sounds emplace a new Sound bound to the pre-loaded SoundBuffer.

4.5 SCORE & HIGH SCORE

Score increments when the player's current Y drops below highestY — correctly counting unique rows crossed rather than total distance. High score persistence uses two inline functions in scoreboard.h operating on highscore.txt. The load function defaults to zero if the file does not exist.

4.6 CAMERA SYSTEM

The SFML View advances cameraY upward at constant cameraSpeed px/sec. If the player falls more than one quarter of the screen height below camera center, the camera snaps to track the player — a hybrid camera: normally auto-scrolling, responsive when the player outpaces it. The view switches back to default for HUD rendering so score text remains screen-fixed.

5. DESIGN PATTERN APPLICATION

5.1 PROTOTYPE PATTERN

The Prototype creational pattern (Gamma et al., 1994) is applied in WorldGenerator to avoid the overhead and coupling of constructing obstacles from scratch on every spawn event.

PROBLEM BEING SOLVED

- ▶ Spawn methods originally constructed obstacles with hardcoded constructor arguments scattered across three different methods.
- ▶ As obstacle types grew (three Vehicle variants, two static types, Log), maintaining construction calls became fragile.
- ▶ Any change to a constructor signature required updates in multiple places — a maintenance liability.

STRUCTURE & IMPLEMENTATION

- ▶ A pure virtual clone() method declared in Obstacle and overridden in every subclass as a one-liner using the copy constructor.
- ▶ WorldGenerator holds a std::map<string, Obstacle*> prototype registry pre-populated in registerPrototypes() with one instance per entity type.
- ▶ spawnFromPrototype(key, x, y) clones the matching prototype and sets its world position — spawn logic requires zero constructor knowledge.
- ▶ The WorldGenerator destructor iterates the map and deletes all prototypes; a move assignment operator prevents double-deletion on game restart.

BENEFITS ACHIEVED

- ▶ Decoupling — spawnVehicles() calls spawnFromPrototype("bike", x, y); constructor parameters are invisible to spawn logic.
- ▶ Extensibility — adding a new entity type (e.g., train) requires only one new prototype registration. Zero changes to spawn methods.
- ▶ Clean lifecycle — all prototype memory is managed deterministically in the WorldGenerator destructor.

5.3 SINGLETON-ADJACENT PATTERN — STATIC COLLISION UTILITY

The Collision class is implemented as a pure static utility class — all methods static, no instance state or constructor. This provides a named, organized namespace for related algorithms without requiring instantiation. main.cpp calls Collision::checkVehicleHit(...) directly, making the dependency explicit and the class easily testable in isolation.

6. LIMITATIONS & FUTURE WORK

6.1 CURRENT LIMITATIONS

#	LIMITATION	TECHNICAL DETAIL	RISK
1	Raw pointer management	vector<Obstacle*> relies on manual new/delete. An exception between spawn and cleanup leaks all obstacle memory. Smart pointers (unique_ptr<Obstacle>) eliminate this.	Medium
2	No difficulty scaling	Vehicle speed and obstacle density fixed regardless of score. The game does not become harder as the player progresses.	Low (Gameplay)
3	Static obstacle overlap	Two trees/rocks can spawn at the same grid X since positions are independently randomized. A tile occupancy map would prevent overlap.	Low
4	No river current	River lanes apply no lateral drift between logs. A constant small force would add difficulty and realism.	Low
5	Single-frame vehicle speed	At very low frame rates, a fast vehicle could tunnel through the player in one frame. A swept-AABB check would fix this edge case.	Low (Edge Case)
6	No pause state	The FSM has no PAUSED state. Adding one requires only a new enum value and an update guard.	Low

6.2 PROPOSED IMPROVEMENTS

- Upgrade to Full State Pattern — Replace GameState enum with polymorphic state classes (IGameState → PlayingState, MenuState, DeadState, PausedState). State transitions become compile-safe and new states are trivially addable.
- Train Obstacle with Nested State Pattern — A Train alternating between MovingState and StoppedState demonstrates nested State pattern application and adds gameplay variety.
- Difficulty Progression System — A difficulty multiplier scaling with score milestones, applied to cameraSpeed, vehicle speed ranges, and obstacle density thresholds.
- Upgrade to Smart Pointers — Replace all raw Obstacle* with unique_ptr<Obstacle> to eliminate manual delete calls and prevent double-deletion bugs.
- Leaderboard with File-Based Persistence — Replace the single highscore.txt integer with a ranked top-10 list displayed on a dedicated LEADERBOARD state accessible from the menu.

7. TEAM CONTRIBUTION

ROLL NO.	NAME	MODULE	DETAILED CONTRIBUTION AREA
CT-25055	Zuha Azhar	SFML Window, Player & Camera	Vehicles (3 types, wraparound); WorldGenerator class; collision (checkVehicleHit, checkDrowning); gap fix; Prototype design pattern; game state; score & high score (file handling)
CT-25070	Areeba	Vehicles, World Gen & Collision	Setup compiler (compile.bat); spawn logs; player rides log; designed sprites; collision (checkStaticBlocking, getLogUnderPlayer); report documentation & design
CT-25087	Muhammad Ashar Hussain	Game Loop, UI & Audio	SFML 3 window setup; Obstacle base class; player movement & boundary checks; camera system; sprite system; title screen design; audio (background music, hop sound, death sound)
CT-25098	Ali Hasan Farooqui	Static Obstacles, UI & UML	Static obstacles (Tree & Rock); UI (pixel font, score & high score HUD, Press ENTER prompt on menu screen); designed UML diagram; provided facts for report

8. WHY WE CHOSE CROSSY ROAD

Crossy Road was selected as the project game for several compelling reasons that align directly with the goals of CT-260 Object Oriented Programming.

Natural OOP Mapping

The game world maps directly and non-artificially to OOP concepts. Vehicles, logs, trees, and rocks are real-world entities with distinct identities, attributes, and behaviors — a perfect fit for class hierarchies and polymorphism. Every concept from the CT-260 syllabus (encapsulation, inheritance, polymorphism, abstraction, composition, dynamic binding) finds a concrete and meaningful application in the codebase rather than a forced or theoretical one.

Well-Defined Scope

Crossy Road has a well-understood, bounded feature set. The core mechanics — player movement, obstacle avoidance, river crossing, and scoring — are clear and implementable within a semester. This made it possible to plan the architecture upfront, divide work cleanly across four team members, and deliver a complete, playable product rather than an unfinished prototype.

Design Pattern Opportunities

The game naturally motivates multiple design patterns. The need to spawn many obstacle instances efficiently leads to the Prototype pattern. The need to cleanly separate menus, gameplay, and death screens leads to the Game State pattern. These patterns arise organically from the problem rather than being imposed artificially to satisfy a checklist.

Familiarity and Motivation

All four team members were familiar with the original Crossy Road mobile game, providing immediate intuition about expected behavior and edge cases. Reimplementing a known game also gave us a clear target for correctness — the game either feels right or it does not — which proved more motivating than building an abstract system with no tangible output to test against.

9. VS CODE PROJECT FILE STRUCTURE

The project is developed in Visual Studio Code and organized into the following directory structure. Every source file is in the root folder for simplicity; assets and build artifacts are separated into dedicated subdirectories.

PATH / FILE	PURPOSE
crossy_road/	Project root — opened as workspace in VS Code
├─ main.cpp	Entry point — game loop, event handling, state machine, camera, HUD
├─ Obstacle.h / .cpp	Abstract base class for all entities; defines virtual update/draw/clone interface
├─ Vehicle.h / .cpp	Vehicle subclass — Bike, Car, Dumper variants via VehicleType enum; horizontal wrap movement
├─ Log.h / .cpp	Log subclass — moving river platform; exposes getSpeedX() for player-riding logic
├─ Tree.h / Rock.h	Static obstacle subclasses — empty update(); provide correct runtime types for Collision's dynamic_cast
├─ Lane.h / .cpp	Single horizontal strip — SAFE / ROAD / RIVER type; drawn as colored rectangle
├─ WorldGenerator.h / .cpp	Procedural generation subsystem — prototype registry, spawn/despawn logic, safety-lane enforcement
├─ Collision.h / .cpp	Static utility class — four collision algorithms: static blocking, vehicle hit, log riding, drowning
├─ AudioManager.h / .cpp	SFML 3 audio wrapper — loads buffers once, plays named sounds via std::optional<Sound>
├─ scoreboard.h	Header-only high score I/O — loadHighScore() and saveHighScore() using highscore.txt
├─ compile.bat	Windows batch script — compiles all sources with g++, links SFML 3 libraries, copies DLLs to output
├─ assets/images/	Sprite sheets and textures for player, vehicles, log, tree, rock, and background
├─ assets/sounds/	Audio files — background music, hop sound effect, death sound effect (.ogg / .wav)
├─ SFML/	SFML 3 SDK directory — include/ headers and lib/ static link libraries (.a files)
├─ highscore.txt	Persistent high score file — created on first run; single integer value
└─ .vscode/	VS Code workspace config — tasks.json (build task), c_cpp_properties.json (IntelliSense include paths)

10. WHY WE CHOSE SFML 3

SFML 3 (Simple and Fast Multimedia Library) was selected as the graphics and audio framework for this project after evaluating several alternatives. The decision was driven by the following factors:

C++ Native Integration

SFML is a C++ library designed specifically for C++ projects. It uses standard C++ classes, namespaces, and object-oriented patterns throughout its API — making it a natural extension of the codebase rather than a foreign abstraction layer. There is no language boundary to cross, no bindings to maintain, and no ABI compatibility concerns.

Appropriate Abstraction Level

SFML sits at the right level of abstraction for a course project. It provides windowing, sprite rendering, texture management, audio playback, and event handling — exactly what is needed — without the overhead of a full game engine (Unity, Unreal) that would hide the OOP architecture we were required to design ourselves. Using a full engine would have reduced the project to scripting rather than software design.

SFML 3 Specifically: Modern C++ API

SFML 3.0 (released January 2024) overhauled the API with modern C++17 conventions. The most impactful change for this project was the Sound class: SFML 3 requires a Sound to be associated with a SoundBuffer at construction rather than allowing late binding. This enforced correct resource management in our AudioManager. The use of `std::optional<Sound>` to defer construction until a buffer is available was a direct consequence of this SFML 3 design decision, and it taught the team an important pattern for managing objects with mandatory dependencies.

Documentation and Community Support

SFML has comprehensive official documentation, a large community of student and hobbyist developers, and extensive tutorial coverage specifically targeted at academic C++ projects. When integration issues arose — particularly around DLL linking on Windows and the SFML 3 audio API changes — solutions were consistently findable, reducing time lost to build-environment debugging rather than application logic.

11. HOW THE .BAT FILE COMPILES THE PROJECT

Because the project targets Windows and does not use a build system such as CMake or Make, a Windows batch file (compile.bat) handles the entire compilation and linking pipeline. Running it from the project root produces a ready-to-execute binary in a single step. The script performs the following stages in order:

Stage 1 — Locate the Compiler

The script sets a variable pointing to the MinGW-w64 g++ executable. All team members install MinGW to the same path to ensure the script runs consistently across machines without modification.

Stage 2 — Set Include and Library Paths

Two paths are passed to the compiler: `-I SFML/include` adds the SFML 3 header directory to the include search path so `#include <SFML/Graphics.hpp>` resolves correctly, and `-L SFML/lib` points the linker to the compiled SFML static library (.a) files bundled inside the project. Keeping the SFML SDK inside the project folder means the build works on any machine without a system-wide SFML installation.

Stage 3 — Compile All Source Files

A single g++ invocation lists every .cpp file explicitly: main.cpp, Obstacle.cpp, Vehicle.cpp, Log.cpp, Lane.cpp, WorldGenerator.cpp, Collision.cpp, and AudioManager.cpp. The `-std=c++17` flag enables C++17 features required by both the codebase and SFML 3. The `-O2` flag enables optimizations for release performance. Because all files are passed together in one command, g++ compiles and links in one step — no separate object file (.o) compilation phase is used.

Stage 4 — Link SFML Libraries

The linker flags connect the compiled code to the SFML 3 modules actually used. The flags passed are `-lsfml-graphics`, `-lsfml-window`, `-lsfml-audio`, and `-lsfml-system`. Order matters: graphics and window depend on system, so system is listed last. On Windows, additional system libraries `-lopengl32`, `-lwinmm`, and `-lgdi32` are linked to satisfy SFML’s platform dependencies. The `-mwindows` flag suppresses the console window so only the SFML render window appears when running the game.

Stage 5 — Output and DLL Copy

The `-o crossy_road.exe` flag names the output binary. After the compiler exits successfully, the batch file copies all required SFML runtime DLLs (sfml-graphics-3.dll, sfml-window-3.dll, sfml-audio-3.dll, sfml-system-3.dll, and the OpenAL DLL opengl32.dll) from SFML/bin/ to the project root alongside the executable. This ensures the game launches correctly when crossy_road.exe is double-clicked, since Windows searches for DLLs in the same directory as the executable first.

Summary of Key Compiler Flags

FLAG	EFFECT
<code>-std=c++17</code>	Enable C++17 for <code>std::optional</code> , structured bindings, and SFML 3 headers
<code>-I SFML/include</code>	Add SFML header directory to include search path
<code>-L SFML/lib</code>	Add SFML library directory to linker search path
<code>-lsfml-graphics</code> etc.	Link SFML module libraries in dependency order (system last)
<code>-mwindows</code>	Suppress the console window; only the game window is shown
<code>-O2</code>	Enable compiler optimizations for release-quality frame rate
<code>-o crossy_road.exe</code>	Name the output executable

12. REFERENCES

- [1] SFML Development Team. (2024). SFML 3.0 API Reference. <https://www.sfml-dev.org/documentation/3.0/>
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.
- [3] Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley Professional.
- [4] NED University of Engineering & Technology. (2026). CT-260: Object Oriented Programming — Course Notes and CCP Guidelines. Department of Computer Science & IT.
- [5] Meyers, S. (2014). Effective Modern C++. O'Reilly Media.
- [6] Gregory, J. (2018). Game Engine Architecture (3rd ed.). CRC Press.

APPENDIX A — SCREENSHOTS

